

Substring :

Subarray (Array) $\longrightarrow$ Substring (string)

- Contiguous part of string
- Complete string is its own substring
- Single character is also a substring

Palindrome



$\begin{matrix} e & s \\ \downarrow & \downarrow \end{matrix}$
N I T I N ✓

Check if a substring of a given string is palindrome?

$\begin{matrix} s & e \\ \downarrow & \downarrow \end{matrix}$
C I V O I C ✗

```
bool isPal ( str[N], s, e) {  
    while (s <= e) {  
        if (str.charAt(s) == str.charAt(e)) {  
            s++;  
            e--;  
        }  
        else {  
            return false;  
        }  
    }  
    return true;  
}
```

TC: $O(N)$

SC: $O(1)$

Amazon Q Given a string return the length of longest palindromic substring. ($1 \leq N \leq 10^3$)

s: a b a c a b

aba → 3

aca

bacab

s: a b a b → 3

s: a b n c → 1

s: a b b a → 4

Brute Force : Iterate over all possible substrings:
 → Check if the substring is palindrome
 → update max length.

for every substring s: → $O(N^2)$ $\left\{ \frac{N(N+1)}{2} \right\}$
 if s is palindrome: → $O(N)$
 ans = max(ans, s.length)

TC: $O(N^3)$

SC: $O(1)$

- Odd length palindromes (center: character)
 - Take every character as a center
 - Calculate longest palindrome around it
 - update the max
- Even length palindromes
 - Take every adjacent character pair as center
 - Calculate longest palindrome around it
 - update the max

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
 x b d y z z y d e b d y z y d x
 ↑ P₁ ↑ P₂

Cum Len : 1 → 5

ans : 1 → 5

(P₁, P₂)

[P₁, P₂] ⇒ P₂ - P₁ + 1

[P₁, P₂) ⇒ P₂ - P₁

(P₁, P₂) ⇒ P₂ - P₁ - 1

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
 x b d y z z y d e b d y z y d x
 ↑ P₁ ↑ P₂

int longPal (S[N]) {

ans = 0;

for (i = 0; i < N; i++) { → O(N) // odd length Palindrome
 P₁ = i; P₂ = i; (TC: O(N²))

while (P₁ ≥ 0 && P₂ < N) { → O(N²)

if (S.charAt(P₁) == S.charAt(P₂)) {

P₁--; P₂++;

}

else { break; }

}

ans = max(ans, (P₂ - P₁ - 1));

}

for (i = 0; i < N; i++) {

P₁ = i; P₂ = i + 1;

while (P₁ ≥ 0 && P₂ < N) {

if (S.charAt(P₁) == S.charAt(P₂)) {

P₁--; P₂++;

}

else { break; }

}

ans = max(ans, (P₂ - P₁ - 1));

}

return ans;

}

// even length Palindrome
 TC: O(N²)

TC: O(N²) SC: O(1)

Facebook
Amazon

Q Given a string, Find length of longest substring with all distinct characters.

s_1 : a b c a b c d d $\rightarrow 4$

s_2 : s i p p i e r g $\rightarrow 5$

s_3 : a a a a a $\rightarrow 1$

Brute Force

For all substrings: $\rightarrow O(N^2)$

check if it contains distinct characters. $\rightarrow O(N)$
using Set

TC: $O(N^2)$

SC: $O(N)$ {Hash Set}

Idea 2

For every index i :

$\rightarrow$ Find the largest substring with distinct characters starting at index i

int distinctCharSubstring (S[N]) {

for (i=0; i<N; i++) {

HashSet <Char> hs;

for (j=i; j<N; j++) {

if (hs.contains (S.charAt(j)))

{ hs.add (S.charAt(j)); }

else { break; }

}

ans = max (ans, hs.size());

}

}

TC: $O(N^2)$

SC: $O(N)$

a a a a a a a b
↓ ↓ ↓ ↓

⋮

0 1 2 3 4 5 6 7 8 9 10 11 12 13
 a b c g h e g k l m n a b k
 ↑ ↑
 P₁ P₂

P ₁	P ₂	set	length	ans
0	0	{a}	1	1
0	1	{a, b}	2	2
0	2	{a, b, c}	3	3
0	3	{a, b, c, g}	4	4
0	4	{a, b, c, g, h}	5	5
0	5	{a, b, c, g, h, e}	6	6
0	6	→ Duplicate		
1	6	{ b, c, g, h, e}		
2	6	{ c, g, h, e}		
3	6	{ g, h, e}		
4	6	{ h, e, g}	3	6
4	7	{ h, e, g, k}	4	
4	8	{ h, e, g, k, l}	5	
4	9	{ h, e, g, k, l, m}	6	
4	10	→ Duplicate		
5	10	{ k , e, g, k, l, m}	5	
5	11	{ e, g, k, l, m, h }	6	

```

int distinctCharSubString ( S[N] ) {
    ans = 0;
    HashSet<Char> hs;
    P1 = 0; P2 = 0;
    while ( P2 < N ) {
        if ( hs.contains ( S.charAt ( P2 ) ) == false ) {
            hs.add ( S.charAt ( P2 ) );
            ans = max ( ans, hs.size() );
            P2 ++;
        }
        else {
            hs.remove ( S.charAt ( P1 ) );
            P1 ++;
        }
    }
    return ans;
}

```

TC : $O(N)$

SC : $O(1)$

Max size of hash set $\longrightarrow 26$

$\longrightarrow$ Constant Space